

TESTs - Lesson 19 - Randomness

Why is generating true randomness a significant challenge on blockchains like the Ethereum Virtual Machine (EVM)?

- Precise hashing and bytes conversion
- Creating a random number from block hash
- Mining exploration
- False randomness

Code references

Code references

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract Randomness {

function getRandomNumber() public returns (uint256) {

A Because it requires too much computational power and gas.

B Because blockchains are deterministic systems that require all nodes to reach the same result.

C Because the Solidity programming language does not have built-in functions for randomness.

What is a potential security risk of using random numbers generated by most standard programming languages for a high-stakes application like a lottery?

- Mining exploration
- False randomness

Code references

Code references

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract Lottery {

function getRandomNumber() public returns (uint256) {

A The number generation process might fail unexpectedly.

B It can be exploited if the generation method is predictable.

C It significantly increases the transaction cost (gas fee).

If a truly random function that produces different results on different nodes were added to the EVM, what fundamental principle of blockchain would it violate?

- Mining exploration
- False randomness

Code references

Code references

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract Blockchain {

function getRandomNumber() public returns (uint256) {

A Immutability

B Consensus

C Transparency

Which of the following patterns for generating pseudo-randomness on-chain is discussed in the video?

- Mining exploration
- False randomness

Code references

Code references

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract PseudoRandomness {

function getRandomNumber() public returns (uint256) {

A Atmospheric Noise Generation

B Black Hash PRNG (Pseudo-Random Number Generation)

C Quantum State Sampling

In the 'commit-reveal' scheme, what does a user do during the 'commit' phase?

- Mining exploration
- False randomness

Code references

Code references

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0; contract CommitReveal {

function commit(uint256 secret) public {

A They reveal their secret number to the smart contract.

B They submit a hash of their secret number plus a random salt.

C They wait for other users to submit their numbers first.

The video discusses several workarounds for on-chain randomness. Which method involves retrieving a random value from an external, off-chain source?

A Black Hash PRNG

B Oracle RNG

C Collaborative PRNG

What is the primary purpose of the 'reveal' phase in a commit-reveal scheme?

A To allow participants to change their initial choice.

B To publicly show the original secret choice and salt, allowing for verification against the previously committed hash.

C To pay the gas fee for participating in the game.

In a collaborative randomness scheme where a user provides a signed 'seed', what does the signature prove?

A That the seed is a truly random number.

B That the user who provided the seed is the same one who signed it, preventing impersonation and manipulation.

C That the seed has been approved by the network validators.

How can the manipulability of on-chain randomness sources like 'blockhash' be reduced when combined with a user-provided seed?

A By requiring the user to wait a certain number of blocks before revealing their seed, they cannot know what the future blockhash will be.

B By asking the user to provide the blockhash themselves.

C By only using the first few digits of the blockhash.

What is the core conflict that makes generating unpredictable random numbers natively on a blockchain so difficult?

A The high cost of gas for complex calculations.

B The limited storage capacity of a block.

C The need for unpredictable results (randomness) conflicts with the need for deterministic execution (consensus).

Why is using a future `blockhash` as a source of randomness problematic for a smart contract?

A

A miner or validator can manipulate the block's contents to influence the hash and gain an advantage.

B

Future block hashes are always zero, making them useless.

C

It is impossible for a smart contract to know what a future block hash will be.